SmartShield

Group 2 Product Requirements Document

Attack Path Validation, Controlled Simulation, Visualization & AI Explanation

Version	1.0
Project	SmartShield - AI-Driven Smart Contract Security Analysis
Group 2 Scope	Delegatecall Attack Path Validation & Product Layer
Primary Goal	Validate candidate attack paths using execution evidence, then safely simulate and explain confirmed findings.

Core principle: Group 2 does not decide vulnerability from the mere presence of delegatecall. It validates candidate paths produced by Group 1 using runtime read/write, storage, reachability, and satisfiability evidence.

1. Purpose

Group 2 owns the second half of SmartShield's delegatecall security pipeline. Group 1 produces candidate delegatecall attack paths from static analysis. Group 2 determines whether those paths are actually executable and whether the expected state/storage behavior occurs.

- Verify whether the candidate execution path is reachable.
- Verify whether the expected vulnerable state variable/storage location is written.
- Verify whether the same relevant state is subsequently read where required.
- Compare predicted storage slot/offset information with runtime observations.
- Check whether relevant path constraints are satisfiable.
- Confirm or reject the candidate vulnerability.
- Safely demonstrate confirmed results in an isolated environment.
- Visualize the attack path and produce a grounded AI explanation/security report.

2. Scope Boundary

Group 2 starts from a CandidateAttackPath object plus the contract/source/bytecode required for validation.

In scope

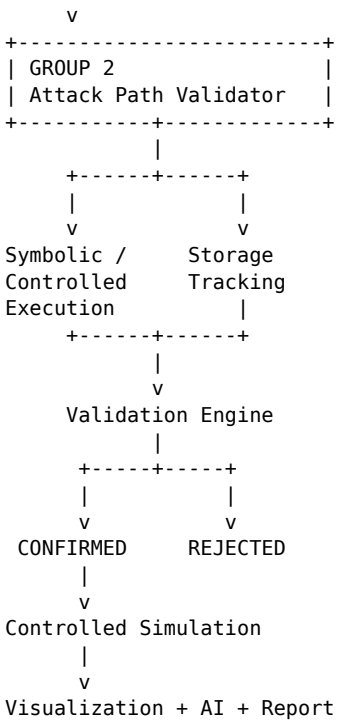
- Contract compilation for the validation environment.
- Integration with an existing EVM symbolic execution/analysis framework.
- Candidate-path execution and constraint validation.
- Runtime storage READ/WRITE capture.
- Storage slot/offset comparison.
- Sensitive-operation reachability checks.
- Confirmed/rejected result generation.
- Controlled local/test simulation.
- Attack-path visualization, API, report, and AI explanation.

Out of scope for Group 2 V1

- Solidity AST generation and full static-analysis pipeline.
- Initial candidate attack-path discovery.
- Building an EVM or symbolic executor from scratch.
- Attacking live contracts, mainnet wallets, or real funds.
- Full blockchain-wide scanning.
- Universal support for every Solidity/EVM version and complex storage structure.
- Unrelated vulnerability families such as reentrancy or integer overflow.

3. High-Level Architecture

```
GROUP 1
Static Analysis
     |
     v
CandidateAttackPath
     |
```



4. Interface from Group 1

Group 2 must not depend on Group 1's internal code. The integration boundary should be a stable JSON/API model.

```
{
  "contract": "Wallet",
  "caller_contract": "Wallet",
  "callee_contract": "WalletLibrary",
  "delegatecall_function": "fallback",
  "vulnerability_type": "ETHER_TRANSFER",
  "attack_path": ["initialize", "execute"],
  "vulnerability_variable": "owner",
  "writer_function": "initialize",
  "reader_function": "execute",
  "expected_storage": {
    "slot": 0,
    "offset": 0
  }
}
```

5. Functional Requirements

FR-01 - Contract Compilation

Accept the Solidity source needed for validation and compile it into bytecode compatible with the chosen controlled execution framework. Record the compiler version. Compilation errors must return an explicit validation failure reason rather than silently continuing.

FR-02 - Symbolic/Controlled Execution Engine

Integrate an existing EVM symbolic execution or equivalent controlled execution framework. Do not build an EVM from scratch. The implementation should prioritize candidate-path validation rather than unrestricted exploration of the complete state space.

- Execute bytecode in an isolated EVM environment.
- Represent relevant transaction inputs/state symbolically where supported.
- Explore conditions needed by the candidate path.
- Determine SAT/UNSAT/UNKNOWN where appropriate.

- Capture storage access and sensitive operations.

FR-03 - Candidate Path Execution

For each CandidateAttackPath, attempt to execute the relevant function sequence and report whether the expected path is executable.

```
Candidate:
initialize() -> execute()

Result:
{
  "execution_status": "EXECUTABLE"
}
```

FR-04 - Runtime READ/WRITE Capture

During execution, capture storage operations with sufficient context to compare them with Group 1's predictions.

```
{
  "operation": "WRITE",
  "slot": "0",
  "offset": "0",
  "function": "initialize"
}

{
  "operation": "READ",
  "slot": "0",
  "offset": "0",
  "function": "execute"
}
```

FR-05 - Storage Validation

Compare expected storage slot/offset information with runtime observations. Report an explicit match or mismatch. For V1, prioritize simple storage layouts and clearly mark unsupported/ambiguous cases instead of guessing.

FR-06 - State Variable Validation

Validate that the writer function performs the expected write and that the reader/sink function subsequently accesses the expected storage location where required.

FR-07 - Constraint Validation

Determine whether the execution conditions required by the candidate path can be satisfied. Use SAT when a valid state/input exists, UNSAT when the analyzed constraints prove the path impossible, and UNKNOWN/TIMEOUT when the engine cannot determine a result within configured limits.

FR-08 - Vulnerability Confirmation

A candidate should be CONFIRMED only when the required evidence is present.

- Candidate path is executable.
- Expected write is detected.
- Expected subsequent read is detected when required by the vulnerability pattern.
- Runtime storage corresponds to the expected vulnerable state.
- Relevant constraints are satisfiable.
- The sensitive operation is reachable.

```
{
  "status": "CONFIRMED",
  "variable": "owner",
  "write_detected": true,
```

```
  "read_detected": true,
  "storage_match": true,
  "constraints": "SAT",
  "sensitive_operation_reached": true
}
```

FR-09 - Rejection / Inconclusive Results

Static candidates that fail validation must not be presented as confirmed vulnerabilities.

```
{
  "status": "REJECTED",
  "reason": "Required path constraints are UNSAT"
}
```

Use an INCONCLUSIVE status for timeout, unsupported bytecode/storage behavior, unresolved callee, or symbolic-execution limitations. Do not convert an analysis failure into a vulnerability.

6. V1 Vulnerability Categories

V1 should align with the three principal anomaly categories used in the DelegateTracker methodology.

Category	Validation objective
Self-destruct	A prior state modification enables/reaches a self-destruct-sensitive path.
Ether/Currency Transfer	A prior state modification enables/reaches a value-transfer-sensitive path.
Judgment/Condition	A prior state modification changes a relevant require/assert/if/loop decision.

7. Controlled Attack Simulation

After validation, SmartShield may safely demonstrate the confirmed path using a local EVM/test chain/symbolic state. This is a controlled demonstration, not exploitation of a live contract.

```
BEFORE
Owner   = Alice
Balance = 10 test ETH

       |
       | validated delegatecall scenario
       v

AFTER
Owner   = Attacker
Balance = 10 test ETH

Potential consequence:
privileged function -> simulated transfer
```

The simulation view should show: initial state, attacker-controlled input, functions executed, delegatecall step, storage modification, sensitive operation, and final state.

8. Visualization Requirements

Provide a visual graph of the validated path. Each important node should expose evidence on click.

```
Attacker
   |
   v
initialize()
   |
   v
WRITE owner [slot 0]
   |
   v
execute()
   |
```

```
   v
READ owner [slot 0]
   |
   v
Sensitive transfer
```

- Show candidate vs confirmed/rejected status.
- Show vulnerable variable/storage location.
- Show writer and reader functions.
- Show SAT/UNSAT/UNKNOWN result.
- Show sensitive-operation reachability.
- Expose runtime evidence rather than only a severity label.

9. AI Explanation Layer

AI is an explanatory layer, not the source of truth for vulnerability detection. It must receive structured validation evidence and explain only what that evidence supports.

```
Input evidence:
- vulnerability_type: ETHER_TRANSFER
- path: initialize -> execute -> transfer
- variable: owner
- slot: 0
- write_detected: true
- read_detected: true
- storage_match: true
- constraints: SAT

Expected AI task:
Explain why the path was validated, its potential impact,
and recommended remediation without inventing evidence.
```

10. Security Report

- Contract name and compiler version.
- Delegatecall location, caller and callee where available.
- Vulnerability category and validation status.
- Candidate/validated attack path.
- Vulnerable variable and storage slot/offset.
- Runtime READ/WRITE evidence.
- Constraint result.
- Sensitive operation reached.
- Controlled simulation summary.
- Impact explanation and remediation recommendation.
- Limitations/inconclusive analysis notes.

11. API Requirements

POST /validate

```
Input:
{
  "contract": "...",
  "candidate_path": { ... }
}

Output:
```

```
{
  "status": "CONFIRMED | REJECTED | INCONCLUSIVE",
  "validation": { ... },
  "runtime_accesses": [ ... ],
  "storage_matches": [ ... ],
  "attack_path": [ ... ]
}
```

POST /simulate

```
Input:
{
  "contract": "...",
  "validated_path": { ... }
}

Output:
{
  "status": "COMPLETED",
  "before_state": { ... },
  "steps": [ ... ],
  "after_state": { ... },
  "impact": { ... }
}
```

POST /explain

```
Input:
{
  "validation_result": { ... }
}

Output:
{
  "explanation": "...",
  "impact": "...",
  "recommendation": "..."
}
```

12. Core Data Models

Model	Required fields
CandidateAttackPath	contract, caller/callee, delegatecall function, vulnerability type, path, variable, writer, reader, expected sto
RuntimeAccess	operation, slot, offset, function, instruction/value when safely available
ValidationResult	status, executable, write/read detected, storage match, constraints, sensitive operation, reason, runtime a
SimulationResult	initial state, ordered steps, final state, demonstrated impact

13. Non-Functional Requirements

Reproducibility

- Record Solidity compiler version.
- Record EVM/symbolic execution framework version.
- Record validation configuration/timeouts.
- Same input/environment should yield reproducible validation results where the underlying engine is deterministic.

Safety

- Never execute exploit transactions against mainnet or real user wallets.
- Never use real funds for simulation.

- Run simulations in isolated/local/test environments.
- Treat external/untrusted contract input as untrusted data.

Explainability

Every confirmed vulnerability must include the evidence that justified confirmation. A bare 'VULNERABLE' label is insufficient.

14. V1 Limitations

- Prioritize simple state variables and storage layouts.
- Complex dynamic arrays, nested mappings and difficult symbolic states may return INCONCLUSIVE.
- Do not promise every Solidity compiler version.
- Use configured timeouts/path limits to control path explosion.
- Unresolved delegatecall targets may be impossible to validate fully.
- Do not claim zero false positives or zero false negatives.

15. Development Order

Phase	Deliverable
Phase 1	Set up the controlled EVM/symbolic execution environment.
Phase 2	Execute one hard-coded candidate path on a deliberately vulnerable test contract.
Phase 3	Capture storage READ/WRITE operations.
Phase 4	Capture and compare slot/offset information.
Phase 5	Validate one known vulnerable delegatecall scenario end-to-end.
Phase 6	Accept Group 1 CandidateAttackPath JSON.
Phase 7	Support the three V1 anomaly categories.
Phase 8	Add controlled simulation.
Phase 9	Build visualization/dashboard.
Phase 10	Add grounded AI explanation.
Phase 11	Generate the final security report.

16. First Milestone

Before building the full UI, Group 2 must prove one known vulnerable candidate path end-to-end: execute the path, observe the expected WRITE, observe the expected READ, match the storage location, satisfy the relevant constraints, reach the sensitive operation, and return CONFIRMED.

```
Known vulnerable test contract
        |
        v
Known CandidateAttackPath
        |
        v
Controlled / symbolic execution
        |
        v
WRITE detected
```

```
        |
        v
READ detected
        |
        v
Storage match
        |
        v
SAT + sensitive operation reachable
        |
        v
CONFIRMED
```

17. Definition of Done

- Consumes Group 1 CandidateAttackPath input through a documented interface.
- Compiles/loads the relevant contract in a controlled execution environment.
- Captures runtime storage accesses.
- Validates expected writes, reads, and storage locations.
- Checks relevant path constraints and reachability.
- Returns CONFIRMED, REJECTED, or INCONCLUSIVE with evidence.
- Safely simulates a confirmed scenario without real funds/live exploitation.
- Displays an attack-path visualization.
- Generates a grounded AI explanation from structured evidence.
- Produces a readable security report.
- Includes vulnerable and safe regression test contracts.

18. Suggested Repository Structure

```
smartshield-group2/
|
+-- validator/
|   +-- symbolic_executor/
|   +-- path_validator/
|   +-- storage_tracker/
|   +-- constraint_checker/
|   +-- vulnerability_classifier/
|
+-- simulator/
|   +-- sandbox/
|   +-- state_tracker/
|   +-- simulation_engine/
|
+-- api/
|   +-- validate/
|   +-- simulate/
|   +-- explain/
|
+-- ui/
|   +-- attack_path_view/
|   +-- validation_result/
|   +-- simulation_view/
|
+-- reports/
|   +-- security_report/
|
+-- tests/
|   +-- vulnerable_contracts/
|   +-- safe_contracts/
|   +-- validation_tests/
|
+-- README.md
```

19. Instruction for Antigravity

> **Inspect the existing SmartShield repository before creating or replacing modules. Map this PRD onto the current architecture, reuse working components, and implement the smallest end-to-end validation milestone first. Do not start by building a generic AI smart-contract scanner.**

Primary research pipeline:

```
DELEGATECALL CANDIDATE
        |
        v
READ/WRITE DATA-FLOW EXPECTATION
        |
        v
CANDIDATE ATTACK PATH
        |
        v
SYMBOLIC / CONTROLLED VALIDATION
        |
        v
STORAGE VALIDATION
        |
        v
CONFIRMED / REJECTED / INCONCLUSIVE
        |
        v
SAFE SIMULATION + EXPLANATION
```

20. Final Deliverable

Group 2 delivers a validation engine that consumes candidate delegatecall attack paths from Group 1, validates them using controlled execution evidence and storage behavior, safely demonstrates confirmed paths in an isolated environment, and presents the findings through an explainable UI, API, AI explanation, and security report.